PROJECT REPORT ON STUDENT PERFORMANCE PREDICTOR

Machine Learning Theory Project: Multi-Model Evaluation and Deployment

Group Members: Fehmina Fazal, Manahil Mehmood, Aleeza Bibi

Department: Department of Computing & Technology (C&T)

Class & Semester: BS-AI (6th Semester)

Subject: Machine Learning Theory

Tools Used: Python, VS Code, Google Colab, GitHub, Streamlit Cloud

AI Assistance: ChatGPT, Google Gemini

Data Source: Kaggle

1. Executive Summary

This project implements an intelligent analytics and decision-support system designed to monitor and predict student academic outcomes (PASS/FAIL status). Utilizing a targeted academic performance dataset of 300 students from Kaggle, the system implements a robust machine learning workflow. We engineered structural baseline rules to calculate cumulative grade behaviors, handled feature dependencies, and evaluated three distinct mathematical classifiers: Logistic Regression, Decision Tree Classifier, and Random Forest Classifier. The highest performing architecture was packaged and deployed onto Streamlit Cloud, establishing an active connection with GitHub to facilitate live user testing and interactive recommendation logic. Development was accelerated via code scaffolding and troubleshooting assistance from ChatGPT and Google Gemini.

2. Project Objectives

- To identify key behavioral and academic features (e.g., study habits, attendance, assignment and quiz scores) that structurally impact student success.
- To build and evaluate multiple supervised machine learning theory algorithms to predict student outcomes accurately.
- To mitigate data risk and operational failures by writing clean preprocessing functions handling textual feature variations.

- To deploy an interactive, production-ready web application enabling instructors to perform real-time batch queries or predict outcomes for individual newly-added student profiles.

3. Dataset Feature Architecture

The academic matrix consists of 300 granular student profiles. Below is a structured summary of the key computational features analyzed:

Feature Name	Data Type	Operational Concept & Bounds
student_id	Categorical (String)	Unique token assigned per student registry.
study_habits	Categorical (Text)	Behavioral feature profiling study routine metrics (e.g., Good, Poor, Excellent, Average).
attendance	Numeric (Decimal)	Percentage of total academic lectures attended by the student.
assignment1, 2, 3	Numeric (Decimal)	Continuous numeric marks indicating assignment grades scored out of 100.
quiz1, 2	Numeric (Decimal)	Continuous numeric scores representing short quiz assessments out of 50.
midterm	Numeric (Decimal)	The principal mid-semester structured theoretical examination score.
avg_score	Numeric (Calculated)	Derived continuous statistical mean calculated across all core graded assignments and quizzes.
Result	Binary Target (0 / 1)	Generated binary flag separating student success (1 for PASS [Avg ≥ 50], 0 for FAIL).

4. Technical Methodology & Machine Learning Workflow

Step 4.1: Feature Engineering and Label Normalization

The dataset was loaded via `pandas` into the development pipeline. Textual categorical columns such as student behavior, gender, and background variables were mapped systematically using `LabelEncoder` routines. A custom algebraic mapping step was introduced to transform the non-numeric target variables into uniform computational matrices, splitting inputs safely into independent features matrix (`X`) and target vectors (`y`).

Step 4.2: Theoretical Model Training & Evaluation Split

To evaluate the model's capacity to generalize on unseen data, the records were split via a deterministic 80% Training set and 20% Testing partition. Three core theoretical classification architectures were initialized and compared:

- **Logistic Regression:** Applied as a baseline linear probabilistic classifier optimized with a maximum iteration threshold of 1000 steps.
- **Decision Tree Classifier:** Deployed to track logical decision structures based on feature impurity parameters.
- **Random Forest Classifier:** Configured as an ensemble tree structure utilizing bootstrapped estimators to eliminate variance and produce stable predictions.

5. Experimental Results and System Insights

- **Model Comparison:** The ensemble tree system (Random Forest) achieved superior prediction scores compared to basic linear strategies, validating complex interactions between midterms and quizzes.
- **Automated Feedback Trigger:** The script embeds an intelligent post-prediction feedback algorithm. If a profile falls below critical score thresholds or triggers a failure state, the application outputs actionable academic recovery protocols.
- **Primary Drivers:** Continuous analysis highlighted that continuous cumulative assignment marks and attendance margins serve as the highest indicator parameters within the tree splits.

6. Cloud Application & Deployment Strategy

The collaborative workflow followed an integrated continuous deployment loop to expose the model as an active system:

1. **Interactive GUI Design (`project.py`):** Built using the Streamlit engine, featuring custom state management to display student metric data tables, filter search bars by student IDs, and dynamic input fields.
2. **Version Controlled Repositories:** Code files, including the structural dataset, were committed and managed on a shared GitHub repository.
3. **Dependency Mapping:** Configured a `requirements.txt` layout containing package instructions for cloud deployment.
4. **Active Host Deployment:** Connected the GitHub branch repository to Streamlit Cloud. The remote host successfully parsed the package manifests, rendering a public dashboard live on the internet.

7. Collaborative & AI Contributions

The execution of this project reflects cross-team execution augmented by modern AI engineering models:

- **Team Split:** Fehmina Fazal, Manahil Mehmood, and Aleeza Bibi jointly handled data gathering, algorithm testing, UI configuration, and documentation.
- **AI Scaffolding:** ChatGPT and Google Gemini were strategically leveraged to format multi-model comparison structures, optimize predictive conditional logic loops, and debug Streamlit file parsing errors.

8. Conclusion

The Student Performance Predictor system successfully establishes a framework for early academic risk evaluation. By combining theoretical machine learning algorithms with automated remediation logic and an accessible cloud dashboard, the system serves as an efficient tool for academic monitoring. The workflow demonstrates a production-grade transition from static datasets to cloud software engineering.